

# MIAM (ICM1A-ICM2A), UP2, TP1 : Identification

Ce travail pratique numérique sera l'occasion d'implémenter une méthode d'identification, et de l'appliquer à l'inférence de la perméabilité inhomogène dans un milieu fibreux à-partir de mesures de flux entrants. Ce TP utilise les résultats du TD d'identification.

**Ce TP est évalué.** Le travail est à faire individuellement. Il faudra déposer, au plus tard deux semaines après la fin de la séance « contrôle » (dernière séance de TP), le code que vous aurez fait, ainsi qu'un compte-rendu sous format **pdf**. Alternativement, il est possible de rendre ce TP sous la forme d'un fichier Jupyter **ipynb**. **Attention :** dans ce cas, il vous est demandé de fournir ce fichier aux formats **ipynb** (pour exécuter les codes) et **pdf** (pour automatiser l'utilisation de Compilatio).

**La forme est importante.** Dans un contexte « ingénieur », un code mal commenté, mal organisé ou avec des noms de fonctions ou de variables mal choisis a vocation à être une source de stress et d'erreurs pour les contributeurs qui s'y pencheront après vous. Dans le contexte présent, un tel code a simplement vocation à être une source de points en moins pour vous. Il en va de même pour le compte-rendu.

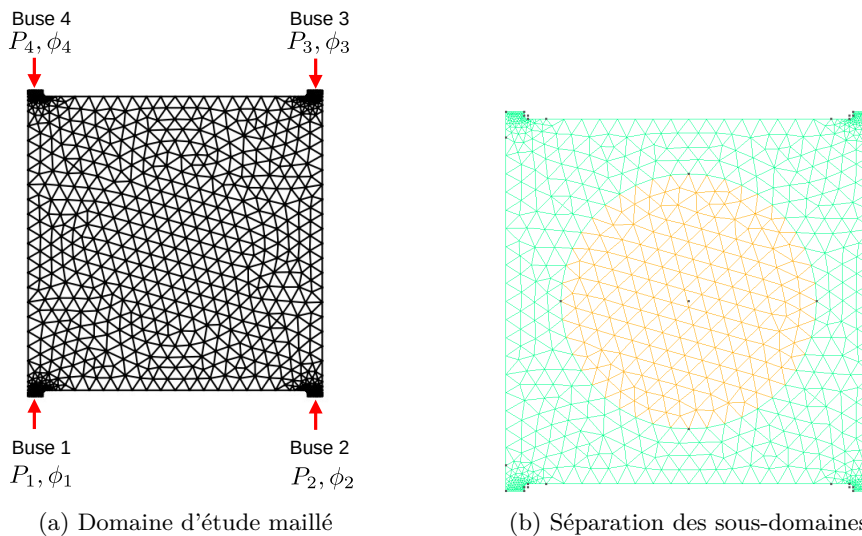
**Forme attendue pour le compte-rendu :** Il doit pouvoir se lire indépendamment du sujet. Il comprendra une rapide introduction expliquant l'objet d'étude, et surtout présentera les résultats de l'exécution de votre code (images ou matrices si elles sont petites), ainsi qu'une description de ces résultats (paramètres utilisés etc) et un commentaire sur ces résultats. Le compte-rendu est organisé en sections (les mêmes que dans le sujet) et se finit par une conclusion, comme rappelé en fin de document. Si le compte-rendu n'est pas un notebook jupyter, le code ne doit pas y apparaître.

## Mise en place

En plus de ce sujet, vous aurez besoin des résultats du TD d'identification, ainsi que du support de cours pour pouvoir mener à bien le travail demandé. Vous avez également à disposition une bibliothèque de fonctions regroupées dans le fichier `tp_identification_ressources`. Cette bibliothèque devra être importée dans le code que vous créerez, de préférence sous un diminutif bien choisi.

On s'intéresse à la caractérisation de la perméabilité d'un milieu poreux à l'aide de mesures de débit à pression imposée. Le milieu que l'on cherche à caractériser est inhomogène, ce qui fait que l'on a besoin de plusieurs points de mesure.

Pour ce faire, on utilise une géométrie carrée, à laquelle on a adjoint quatre buses d'injection de fluide numérotées de 1 à 4. Cette géométrie, après maillage, est présentée sur la figure 1a.



L'idée est d'imposer une pression de fluide  $P_i$  différente aux quatre buses, de mesurer le débit entrant  $\phi_i$ , et de déduire de cette mesure la perméabilité du milieu. Remarquons qu'en pratique, il est

assez difficile d'imposer exactement la pression en un point. C'est pour cette raison que l'on utilisera en fait deux capteurs par buse. L'un (par exemple piézo-électrique) mesure la pression, et l'autre (par exemple un débitmètre électromagnétique ou à effet Coriolis) mesure le flux entrant.

On rappelle le système d'équations auquel obéit le transfert de fluides dans un milieu poreux, l'équation de Darcy, associée à ses conditions aux limites :

$$\begin{cases} \operatorname{div}(k \underline{\operatorname{grad}}(p)) = 0 & \text{dans } \Omega \\ k \frac{\partial p}{\partial \underline{n}} = V_n & \text{sur } \Gamma_n \\ p = U_d & \text{sur } \Gamma_d \end{cases} \quad (1)$$

On veut identifier la distribution  $k(x)$ . Pour simplifier le problème, on suppose connu le fait que ce champ  $k$  est en fait homogène par morceaux. Le domaine  $\Omega$  se sépare en deux sous-domaines ayant chacun une perméabilité homogène. Le domaine  $\Omega_2$  occupe un disque central tandis que le domaine  $\Omega_1$  lui est complémentaire à  $\Omega$  (voir la figure 1b). On n'a donc à identifier que deux paramètres :  $k_1$  et  $k_2$ .

Remarquons qu'en pratique, il est rare que l'on connaisse la géométrie d'une inclusion et pas sa perméabilité. Il peut arriver plus fréquemment que l'on veuille identifier le champ complet, c'est à dire une valeur différente de  $k$  en chaque point de Gauss (en chaque élément) du domaine discrétisé. un tel problème est plus complexe, et demande notamment d'utiliser une méthode de régularisation adaptée.

## 1 Utilisation du code de calcul par éléments finis

L'équation de Darcy sera résolue à l'aide de la méthode des éléments finis. À cet effet, un petit code dédié vous est fourni dans les données du TP. À la fin du fichier `tp_identification_ressources`, la syntaxe d'utilisation des différentes fonctions de ce code est donnée.

On dispose de deux groupes de deux fichiers dans le répertoire `measurements`. Il s'agit de mesures obtenues dans un cas inhomogène, ou dans un cas homogène (suffixe `_H`). La fonction `readMeasurement` de la bibliothèque de ressources permet d'extraire les données contenues dans ces fichiers, à savoir les objets suivants :

- `phi1` à `phi4`, les flux entrants aux différentes buses.  $\phi_i = LV_i$ , où  $L = 0.02 \text{ mm}$  est la largeur d'une buse et  $V_i$  une vitesse normale.
- `um`, dont on ne va pas se servir (sauf pour déboguer et dans la partie bonus). Il s'agit du champ  $p$  sur tout le domaine. Il est évidemment illusoire de supposer qu'une telle mesure puisse être disponible dans un cas réel (surtout sans erreur de mesure), mais c'est un exercice intéressant d'effectuer l'identification à partir de tout ce champ.
- `Pi`, le projecteur  $\Pi$  tel que  $\Pi p = P$ , où  $P$  est le vecteur de taille 4 regroupant les pressions  $P_i$  en chaque buse.
- `Pm`, le vecteur de mesures, tel que  $P_m = \Pi u_m$ .

**Remarque.** Il existe également une fonction appelée `simulateMeasurement`, qui permet de créer une mesure fictive par le calcul. Sauf imprévu, on n'utilisera pas cette fonction.

**Remarque.** Les mesures `measure_sb2` et `measure_sb2_Pi` sont définies sur le maillage `quad_sb2`. Si vous voulez les utiliser sur le maillage `quad_sb`, il faut utiliser la fonction de ré-interpolation de maillages `meshInterpolate`. Ce n'est pas demandé par défaut.

**À faire :** Extraire les mesures l'aide de la fonction `readMeasurement` et afficher le champ  $u_m$ . Pour info, les perméabilités qui ont permis de générer cette mesure, et qui sont celles que la méthode d'identification devra retrouver sont (1;2).

L'identification sera faite à l'aide du calcul dual, c'est à dire en imposant les conditions de Neumann (les vitesses normales  $V_i$ ) et en utilisant les pressions  $P_i$  pour recalculer les paramètres recherchés. Sur le reste de la surface  $\partial\Omega$ , la vitesse normale est imposée nulle. Numériquement, en éléments finis, imposer

un flux nul ne demande aucune modification de la matrice de raideur ni du second membre (en fait, il s'agit d'ajouter des termes nuls à ce second-membre).

Malheureusement, le problème résultant n'a que des conditions aux limites de type Neumann. La solution d'un tel problème n'est pas unique. En fait, il existe un mode à énergie nul uniforme. Dit autrement, la solution est unique à une constante près. Pour fixer cette constante, on peut par exemple imposer que la moyenne de  $p$  soit nulle. Ceci n'est pas optimal au niveau des propriétés de la matrice de raideur  $K$  résultante (sa *sparsité*). En conséquence, on préfère fixer  $p = 0$  en un point choisi au hasard. Ceci est accompli en fixant à `True` le troisième paramètre (facultatif) de la fonction `DirichletScalar`.

**À faire :** Construire une fonction effectuant ce calcul sur le maillage `quad_sb.msh`, et afficher le résultat. Le comparer avec  $u_m$ . On prendra notamment garde à la valeur de la condition de Neumann (en vitesse normale moyenne) à imposer sur la surface de chacune des buses. Attention : le point auquel la pression a été imposée ne doit pas causer de singularité sur la cartographie. Si tel est le cas, cela signifie que les flux imposés ne sont pas équilibrés. Vous pourrez vous inspirer du `main` du fichier `tp_identification_ressources.py`, mais en modifiant les dictionnaires de Neumann et de Dirichlet.

À ce stade, on s'attend à ce que le résultat obtenu ne corresponde pas vraiment à  $u_m$ . Ceci est dû au fait que l'on a imposé arbitrairement la constante sans prendre en compte la mesure effectuée. Il serait possible d'imposer directement que cette constante ait la bonne valeur dès l'étape du calcul par éléments finis. Dans le cas présent, on va plutôt post-traiter la solution obtenue à l'aide d'une translation qui minimise l'écart à la mesure. Si on note  $p_t = p + t$  le champ de pression transformé, avec  $t \in \mathbb{R}$ , la translation doit être solution du problème suivant :

$$\min_t \frac{1}{2} \|\Pi u_m - \Pi p_t\|^2 = \min_t \frac{1}{2} \sum_{i=1}^4 ([\Pi u_m]_i - [\Pi p]_i - t)^2 \quad (2)$$

En annulant la dérivée de la fonction-coût, on aboutit à :

$$\sum_{i=1}^4 ([\Pi u_m]_i - [\Pi p]_i) - 4t = 0 \quad (3)$$

D'où :

$$p_t = p + \frac{1}{4} \sum_{i=1}^4 ([\Pi u_m]_i - [\Pi p]_i) \quad (4)$$

**À faire :** Implémenter cette translation dans la fonction créée précédemment.

On dispose donc à présent d'une fonction capable d'effectuer un calcul direct en connaissant les flux entrants aux différents bords, ainsi que les paramètres du milieu. C'est cette fonction qui sera appelée itérativement lors de la résolution du problème inverse.

**À faire :** La solution trouvée est-elle parfaitement égale à  $u_m$  ? Pourquoi ? Quelle sera la conséquence sur l'identification ? On pourra notamment afficher la différence entre le champ solution trouvé et le champ de mesure  $u_m$ .

**Remarque.** D'autres maillages sont proposés dans le dossier *meshes*. Il s'agit de conduits de différentes formes sur lesquels vous pouvez faire des calculs si vous en avez envie. Sur ces maillages, les bords 1 et 2 sont les bords d'entrée et sortie du fluide, et le bord 3 contient, lui, tous les éléments de bord.

## 2 Identification par substitution (méthode des sensibilités)

On va identifier les paramètres à l'aide de la méthode des sensibilités, résultant de la prise en compte de la contrainte par la méthode de substitution. Cette méthode est rappelée sur l'algorithme 1.

---

**Algorithme 1** : Identification par la méthode de substitution (algorithme de Newton)

---

```
 $\varepsilon$  paramètre de la méthode (pas différences finies)
 $f$  second-membre du problème discrétisé par éléments finis
 $k \leftarrow k_0$  (initialisation)
for  $i = 1..n_{iter}$  do
    Calculer  $p_0$  la solution de  $K(k)p_0 = f$  (via la fonction de la section 1)
    for  $d = 1..D_{max}$  (boucle sur les dimensions de  $k$ ) do
         $\tilde{k} \leftarrow k$ 
         $\begin{bmatrix} \tilde{k} \end{bmatrix}_d \leftarrow \begin{bmatrix} \tilde{k} \end{bmatrix}_d + \varepsilon$ 
        Calculer  $p_d$  la solution de  $K(\tilde{k})p_d = f$  (via la fonction de la section 1)
         $J_d \leftarrow \frac{p_d - p_0}{\varepsilon}$ 
    end
     $J \leftarrow \begin{pmatrix} J_1 & J_2 & \dots & J_d \end{pmatrix}$  (assembler la matrice Jacobienne colonne par colonne)
     $S = \Pi J$  (matrice de sensibilité des mesures aux paramètres)
     $g \leftarrow S^T (\Pi p_0 - P_m)$  (calcul du gradient)
     $H = S^T S$  (calcul de la Hessienne)
    Résoudre  $H\delta = g$ 
     $k \leftarrow k - \delta$  (mise à jour)
end
Retourner  $k$ 
```

---

## 2.1 Cas à un paramètre

Dans un premier temps, on suppose que la perméabilité du milieu est homogène. La valeur cible de référence, que l'algorithme d'identification est censée retrouver, est  $k_{ref} = 1$ .

**À faire** : Adapter les arguments de la fonction `readMeasurement` à ce cas.

**À faire** : Implémenter l'identification de ce paramètre (en partant d'une valeur fausse). Analyser la courbe de décroissance du résidu (norme de  $\Pi p - \Pi u_m$ ) et de l'erreur (valeur absolue de  $k - k_{ref}$  où  $k_{ref}$  est la valeur « vraie »). **Attention** : les calculs directs  $K(k)p_0 = f$  et  $K(\tilde{k})p_d = f$  doivent être faits via la fonction construite à la section 1. En particulier, il est indispensable que l'opération de translation (4) soit faite.

## 2.2 Cas à deux paramètres

On utilise à présent l'autre jeu de mesures. Les valeurs cibles de référence, que l'algorithme d'identification est censée retrouver, sont  $(k_1^{ref}; k_2^{ref}) = (1; 2)$ .

**À faire** : Implémenter l'identification dans le cas à deux paramètres. Tester différents couples de valeurs, et commenter la précision de l'identification. Tracer et commenter les courbes de décroissance du résidu et de l'erreur (en norme 2).

Pour rendre l'identification plus fiable dans les cas avec un gros écart entre  $k_1$  et  $k_2$ , on peut tenter d'utiliser la régularisation de Tikhonov. Il s'agit de transformer la fonction-coût de la façon suivante :

$$\varphi_\mu(k) = \varphi(k) + \frac{\mu}{2} \|k - k_v\|^2 \quad (5)$$

où  $\mu$  est un paramètre scalaire positif, et  $k_v$  une valeur « vraisemblable » pour  $k$ . Remarquons que dans le cadre de l'UP précédente, on a toujours pris  $k_v = 0$ .

**Bonus** : Montrer que cela revient à changer l'expression de  $H$  et  $g$  de la façon suivante :

$$\begin{cases} H_\mu = S^T S + \mu \mathcal{I} \\ g_\mu = S^T (\Pi p - P_m) + \mu (k - k_v) \end{cases} \quad (6)$$

où  $\mathcal{I}$  est l'identité de taille 2.

**Bonus :** Implémenter cette régularisation, et étudier l'influence de  $\mu$  et  $k_v$  sur la qualité de la solution. En particulier, si on prend  $k_v$  égal à la vraie solution, et  $\mu$  très grand, la méthode donne de « très bons » résultats. Mais en pratique, un tel  $k_v$  ne peut pas être connu à l'avance. On prendra donc des valeurs de  $k_v$  plus réalistes.

### 2.3 Bonus : Identification par le Lagrangien (méthode de l'équation adjointe)

On va utiliser la méthode de l'état adjoint, c'est-à-dire l'algorithme 2. Pour ce faire, on donne la fonction `computeGradient`, capable de calculer le vecteur  $p^T \nabla K \lambda$ .

---

**Algorithme 2 :** Identification par la méthode de l'état adjoint (algorithme du gradient)

---

```

 $\rho$  pas de l'algorithme (appelé aussi taux d'apprentissage)
 $\{\kappa_d\}_{d=1..D_{max}}$  base de l'espace de dimension finie dans lequel on cherche  $k$ 
 $k \leftarrow k_0$  (initialisation)
for  $i = 1..n_{iter}$  do
    Résoudre  $K(k)p = f$  (via la fonction de la section 1)
    Résoudre  $K(k)\lambda = \Pi^T (P_m - \Pi p)$  (via une nouvelle fonction à créer)
     $g \leftarrow p^T \nabla K(k)\lambda$ 
     $k \leftarrow k - \rho g$  (mise à jour)
end
Retourner  $k$ 

```

---

La fonction réalisant le calcul adjoint suit le même modèle que celle de la section 1, mais les modes rigides sont fixés à zéro.

**Bonus :** Construire la fonction permettant de réaliser le calcul adjoint et implémenter l'algorithme du gradient 2.

La méthode, une fois implémentée, donne des résultats peu satisfaisants. On ne parvient à identifier les paramètres de conductivité correctement qu'après un très grand nombre d'itérations. Ceci est dû au très mauvais conditionnement du problème inverse, qui avait déjà été constaté dans la partie précédente. On aurait en fait besoin d'une méthode d'optimisation plus efficace pour espérer aboutir à une convergence plus rapide. Par exemple, la méthode d'optimisation BFGS, associée à une recherche en ligne (*linesearch* assurant le respect des conditions de Wolfe) est un bien meilleur choix pour ce type de problèmes.

**Bonus :** La fonction `searchOnLine` est une fonction de *linesearch*. Utiliser cette fonction pour accélérer la convergence de l'algorithme du gradient.

Une fois le *linesearch* mis en place, l'implémentation de l'algorithme BFGS est relativement faisable en pratique. Mais introduire cet algorithme demanderait quand-même un travail conséquent. Ce n'est pas demandé ici.

**Bonus :** Tester la méthode dans le cas factice dans lequel on est capable de mesurer le champ  $p$  en tout point (c'est à dire en utilisant un sortant de `readMeasurement`). Dans ce cas, le projecteur  $\Pi$  est égal à l'identité. La convergence est-elle meilleure ?

## Conclusion

**À faire :** Proposer une conclusion synthétique, claire et pertinente à ce travail. On s'interrogera notamment (mais pas nécessairement exclusivement) sur les questions suivantes :

- L'importance de la précision du modèle utilisé
- Le fonctionnement de la régularisation
- Des considération de temps de calcul et de performances des méthodes